MPMC — Module 2 · PART 2: 8086 ADDRESSING MODES — EXAM-READY NOTES

Topic: Memory access fundamentals (EA · BA · MA, Segment:Offset, physical address) + all 12 addressing modes

Source: Module 2 lecture slides (ECE3004 – Microprocessors and Microcontrollers).

Module 2 is split into 3 files — use them together, nothing is left out:

- **Part 1** → `MPMC_Module2_Part1_Architecture.md` — generations, pins, BIU/EU, registers, flags, physical address, interrupts
- **Part 2 (this file)** → all 12 addressing modes with worked examples
- **Part 3** → `MPMC_Module2_Part3_Instruction_Set.md` — instruction set, assembler directives, procedures/macros, programs
- (All-in-one master copy: `MPMC_Module2_Exam_Notes.md`)

CONTENTS OF THIS FILE

1. What is an addressing mode
2. Memory access fundamentals — EA, BA, MA and physical address calculation
3. Which registers/combinations are allowed inside [ ]
4. Group I — addressing modes for register and immediate data
5. Group II — addressing modes for memory data
6. Group III — addressing modes for I/O ports
7. Group IV — relative addressing mode
8. Group V — implied addressing mode
9. Master summary table of all 12 modes
10. Most likely exam questions (addressing modes)
11. Last-minute revision cram sheet (addressing modes)

1. WHAT IS AN ADDRESSING MODE?

Definition (slide wording): Every instruction of a program has to operate on a data. **The different ways in which a source operand is denoted in an instruction are known as addressing modes.**

The 12 addressing modes, arranged in the slide's 5 groups:

Group	Addressing modes
Group I — for register and immediate data	1. Register Addressing, 2. Immediate Addressing
Group II — for memory data	3. Direct, 4. Register Indirect, 5. Based, 6. Indexed, 7. Based Index, 8. String
Group III — for I/O ports	9. Direct I/O port, 10. Indirect I/O port
Group IV — relative	11. Relative Addressing
Group V — implied	12. Implied Addressing

2. MEMORY ACCESS FUNDAMENTALS

2.1 The problem

- The 8086 has **20 address lines** → can address up to **2^{20} = 1 M bytes** of memory.
- **However, the largest register is only 16 bits.**
- Therefore the physical address has to be **calculated**.

2.2 Definitions

- **Physical Address:** the **actual address of a byte in memory**, i.e. the value which **goes out onto the address bus** (20 bits).
- Memory address is represented in the form **Seg : Offset** — e.g. **89AB:F012**.
- **16 bytes of contiguous memory = one paragraph** (hence "paragraph to byte" = × 10H in hex).

2.3 The calculation (slide method)

Each time the processor wants to access memory, it **takes the contents of a segment register, shifts it one hexadecimal place to the left** (same as multiplying by 16_{10}), then **adds the required offset** to form the **20-bit address**.

```
Physical Address = (Segment register × 16₁₀) + Offset
```

Worked example — 89AB:F012

```
  Segment  89AB  →  89AB0     (paragraph to byte → 89AB × 10H = 89AB0)
  Offset   F012  →   F012     (offset is already in byte units)
                   -----
  Physical address   98AC2 H  (the absolute address)
```

The adder block diagram (slide figure): a **20-bit adder** takes the **16-bit segment register** on one side (with the low 4 bits as `0000`) and the **16-bit offset value** on the other side to produce the **20-bit physical address**.

2.4 The three quantities used throughout this file

Symbol	Stands for	Formula
EA	**Effective Address** (the offset part)	Depends on the addressing mode
BA	**Base Address**	BA = (Segment register) × 16_{10}
MA	**Memory Address**	MA = BA + EA

For a **word** operation, the address pair is written **(MA ; MA + 1)** — the low byte at MA and the high byte at MA+1.

2.5 Default segment registers for each offset register

Offset register used	Default segment register
BX, SI, DI, 16-bit displacement (disp16)	**DS**
BP	**SS**
SP (stack instructions)	**SS**
IP (instruction fetch)	**CS**
String **destination** (**DI**)	**ES**

In the slides the rule is stated as: "Content of the DS register is used for base address calculation" for register indirect; **BX with DS**, **BP with SS**; string source = **DS:SI**, string destination = **ES:DI**.

3. ALLOWED REGISTER COMBINATIONS INSIDE [ ]

To access memory we use these four registers: **BX, SI, DI, BP**. Combining these registers inside [ ] symbols gives different memory locations (Effective Address, EA).

Supported combinations (exact table from the slides):

With no displacement	With 8-bit displacement (d8)	With 16-bit displacement (d16)
[BX + SI]	[BX + SI + d8]	[BX + SI + d16]
[BX + DI]	[BX + DI + d8]	[BX + DI + d16]
[BP + SI]	[BP + SI + d8]	[BP + SI + d16]
[BP + DI]	[BP + DI + d8]	[BP + DI + d16]
[SI]	[SI + d8]	[SI + d16]
[DI]	[DI + d8]	[DI + d16]
[BX]	[BX + d8]	[BX + d16]
[BP]	[BP + d8]	[BP + d16]
—	—	d16 (variable offset only)

Rule: Base = **BX or BP** (BP → SS); Index = **SI or DI** (both → DS). You may use one base, one index, and one displacement — nothing else.

4. GROUP I — ADDRESSING MODES FOR REGISTER AND IMMEDIATE DATA

4.1 Register Addressing

The instruction specifies the **name of the register which holds the data** to be operated upon.

```
MOV CL, DH        (CL) ← (DH)
ADD AX, BX        (AX) ← (AX) + (BX)
```

- The content of the 8-bit register DH (**source**) is only **copied** to another 8-bit register CL (**destination**).
- **The content of the source will NOT be cleared** after the execution of this instruction.

4.2 Immediate Addressing

An **8-bit or 16-bit data is specified as part of the instruction**.

```
MOV DL, 08H       (DL) ← 08H
SUB AX, 0A9FH     (AX) ← (AX) – 0A9FH
```

The 8-bit data (08H) given in the instruction is moved to DL; the 16-bit data (0A9FH) is subtracted from AX.

5. GROUP II — ADDRESSING MODES FOR MEMORY DATA

5.1 Direct Addressing

- The **effective address of the memory location** at which the data operand is stored is **given in the instruction**.
- The effective address is just a **16-bit number written directly in the instruction**.
- This mode is called **direct** because the **displacement of the operand from the segment base is specified directly** in the instruction.

```
MOV [2000H], 02H
MOV [0234H], 0234H
MOV BX, [1354H]
MOV [1234H], BX
MOV BL, [1354H]
MOV [0400H], CL
```

The square brackets around **1354H** denote the **contents of the memory location**. When executed, `MOV BX, [1354H]` copies the contents of memory location 1354H into the BX register.

5.2 Register Indirect Addressing

- The **name of the register which holds the effective address (EA)** is specified in the instruction.
- Registers used to hold EA: **BX, BP, DI and SI**.
- Content of the **DS register** is used for base address calculation.

```
MOV CX, [BX]
MOV [BX], AL
MOV [BP], 52H
MOV [SI], 1234H
```

Operations for `MOV CX, [BX]`:

```
EA = (BX)
BA = (DS) × 16₁₀
MA = BA + EA
(CX) ← (MA)          or,  (CL) ← (MA)
                          (CH) ← (MA + 1)
```

Note (slide): Register/memory enclosed in brackets refers to the content of the register/memory.

5.3 Based Addressing

- **BX or BP** is used to hold the **base value** for the effective address, and a **signed 8-bit or unsigned 16-bit displacement** is specified in the instruction.
- In case of an **8-bit displacement, it is sign-extended to 16 bits** before adding to the base value.
- When **BX** holds the base value of EA → the 20-bit physical address is calculated from **BX and DS**.
- When **BP** holds the base value of EA → **BP and SS** are used.

```
MOV AX, [BX + 08H]
MOV [BX + 1234H], AX
MOV [BP + 21H], 9878H
```

Operations for `MOV AX, [BX + 08H]`:

```
0008H ← 08H   (sign extended)
EA = (BX) + 0008H
BA = (DS) × 16₁₀
MA = BA + EA
(AX) ← (MA)        or,  (AL) ← (MA)
                        (AH) ← (MA + 1)
```

5.4 Indexed Addressing

- **SI or DI** register is used to hold an **index value** for memory data and a **signed 8-bit or unsigned 16-bit displacement** is specified in the instruction.
- The displacement is added to the index value in SI or DI to obtain the EA.
- In case of an 8-bit displacement, it is **sign-extended to 16 bits** before adding.

```
MOV CX, [SI + 0A2H]
```

```
FFA2H ← A2H   (sign extended)
EA = (SI) + FFA2H
BA = (DS) × 16₁₀
MA = BA + EA
(CX) ← (MA)        or,  (CL) ← (MA)
                        (CH) ← (MA + 1)
```

5.5 Based Index Addressing

- The **effective address is computed from the sum of a base register (BX or BP), an index register (SI or DI) and a displacement.**

```
MOV DX, [BX + SI + 0AH]
```

```
000AH ← 0AH   (sign extended)
EA = (BX) + (SI) + 000AH
BA = (DS) × 16₁₀
MA = BA + EA
(DX) ← (MA)        or,  (DL) ← (MA)
                        (DH) ← (MA + 1)
```

5.6 String Addressing

- **Employed in string operations** to operate on string data.
- The **effective address (EA) of source data is stored in the SI register** and the **EA of destination is stored in the DI register**.
- The segment register for calculating the base address of **source** data is **DS** and that of the **destination** data is **ES**.

```
MOVS BYTE
```

Operations:

```
Calculation of source memory location:
  EA = (SI)      BA = (DS) × 16₁₀      MA = BA + EA

Calculation of destination memory location:
  EAE = (DI)     BAE = (ES) × 16₁₀     MAE = BAE + EAE

(MAE) ← (MA)

If DF = 1 → (SI) ← (SI) − 1  and (DI) ← (DI) − 1
If DF = 0 → (SI) ← (SI) + 1  and (DI) ← (DI) + 1
```

Note: EAE = Effective address of the **Extra segment** register. (For word strings the increment/decrement is 2 instead of 1 — see Part 3, string instructions.)

6. GROUP III — ADDRESSING MODES FOR I/O PORTS

These addressing modes are used to access data from **standard I/O-mapped devices or ports**.

6.1 Direct I/O Port Addressing

In direct port addressing mode, an **8-bit port address is directly specified** in the instruction.

```
IN AL, [09H]
```

```
PORTaddr = 09H
(AL) ← (PORT)        ; content of the port with address 09H is moved to AL
```

6.2 Indirect I/O Port Addressing

In indirect port addressing mode, the instruction specifies the **name of the register which holds the port address**. In the 8086, the **16-bit port address is stored in the DX register**.

```
OUT [DX], AX
```

```
PORTaddr = (DX)
(PORT) ← (AX)        ; content of AX is moved to the port whose address is in DX
```

Compare with memory indirect: memory indirect can use BX/BP/SI/DI, but **I/O indirect uses only DX**.

7. GROUP IV — RELATIVE ADDRESSING MODE

- In this addressing mode, the **effective address of a program instruction is specified relative to the Instruction Pointer (IP)** by an **8-bit signed displacement**.

```
JZ 0AH
```

Operations:

```
000AH ← 0AH      (sign extend)
If ZF = 1, then
    EA = (IP) + 000AH
    BA = (CS) × 16₁₀
    MA = BA + EA
    → the program control jumps to the new address calculated above
If ZF = 0, then the next instruction of the program is executed
```

Used by the whole family of **conditional jumps and LOOP instructions** — see Part 3, control transfer instructions.

8. GROUP V — IMPLIED ADDRESSING MODE

- **Instructions using this mode have no operands.**
- The **instruction itself will specify the data** to be operated on by the instruction.

```
CLC          ; this clears the carry flag to zero (CF ← 0)
```

Same family: STC, CMC, CLD, STD, CLI, STI, NOP, HLT.

9. MASTER SUMMARY TABLE OF ALL 12 MODES

Use this to answer "list and explain the addressing modes with examples":

#	Mode	Group	Example	Where the operand / its address comes from
1	Register	I	`MOV CL, DH`	Inside a register named in the instruction
2	Immediate	I	`MOV DL, 08H`	Data is part of the instruction itself
3	Direct	II	`MOV BX, [1354H]`	16-bit EA written directly in the instruction
4	Register Indirect	II	`MOV CX, [BX]`	EA is held in BX/BP/SI/DI
5	Based	II	`MOV AX, [BX + 08H]`	(BX/BP) + disp (8-bit sign-extended / 16-bit)
6	Indexed	II	`MOV CX, [SI + 0A2H]`	(SI/DI) + disp
7	Based Index	II	`MOV DX, [BX + SI + 0AH]`	(BX/BP) + (SI/DI) + disp
8	String	II	`MOVS BYTE`	Source DS:SI, destination ES:DI, auto ±1/±2 by DF
9	Direct I/O port	III	`IN AL, [09H]`	8-bit port address in the instruction
10	Indirect I/O port	III	`OUT [DX], AX`	16-bit port address in DX
11	Relative	IV	`JZ 0AH`	(IP) + 8-bit signed displacement, base CS
12	Implied	V	`CLC`	Operand implied by the instruction itself

One-line classification trick: for **memory modes**, remember **"Direct = address in instruction; Indirect = address in register; Based = BX/BP + disp; Indexed = SI/DI + disp; Based-Index = both + disp; String = SI/DI with DS/ES."**

Sign-extension rule (very commonly asked): an **8-bit displacement is sign-extended to 16 bits** before adding (so `[BX + 08H]` adds 0008H, while `[SI + 0A2H]` adds **FFA2H** because A2H has its MSB set).

10. MOST LIKELY EXAM QUESTIONS (ADDRESSING MODES)

Question	Answer skeleton
Define addressing mode. List and explain the addressing modes of 8086 with examples.	Start with the definition, then the 5-group table, then explain each of the 12 modes with the slide example and its EA/BA/MA calculation. Finish with the master summary table of §9.
Explain the addressing modes for memory data (Group II).	Direct, Register indirect, Based, Indexed, Based-index, String — each with the worked EA = …, BA = (DS) × 16_{10}, MA = BA + EA chain.
Explain the addressing modes of the 8086 used for I/O ports.	Direct port (8-bit port address in instruction, `IN AL,[09H]`) and Indirect port (**16-bit port address in DX**, `OUT [DX],AX`).
How does the 8086 generate a 20-bit physical address from two 16-bit values?	Segment register shifted left one hex digit (×16_{10}) + offset, using the 20-bit adder; worked example 89AB:F012 = 98AC2H; represent as Seg:Offset.
Which registers can be used inside `[ ]` and in which combinations?	BX/BP (base) and SI/DI (index) only, optionally with d8/d16 — reproduce the combination table of §3.
What is the default segment register for BX / BP / SP / SI / DI / IP?	DS for BX, SI, DI and disp; **SS for BP and SP**; **CS for IP**; **ES for the string destination (DI)**.
Give an example showing sign extension of an 8-bit displacement.	`MOV CX, [SI + 0A2H]` → FFA2H sign-extended, EA = (SI) + FFA2H.
Differentiate direct and register-indirect addressing.	Direct: 16-bit EA written in the instruction (e.g. `MOV BX,[1354H]`). Register indirect: EA held in a register (BX/BP/SI/DI), e.g. `MOV CX,[BX]`.
Which addressing mode do conditional jumps use?	**Relative addressing** — EA = (IP) + 8-bit signed displacement, base address from CS; control transfers only if the flag condition is true.
Write a note on implied addressing mode.	No operands in the instruction; the instruction itself specifies the data (e.g. `CLC` clears CF).

11. LAST-MINUTE CRAM SHEET (ADDRESSING MODES)

The 3 formulas to write first in any answer:

```
EA = Effective Address (offset part)
BA = (Segment register) × 16₁₀
MA = BA + EA                [ for a word: (MA ; MA + 1) ]
```

Physical address:

```
Physical Address = (Segment × 16₁₀) + Offset      e.g. 89AB:F012 → 98AC2H
```

Registers allowed in `[ ]`: BX, BP, SI, DI (+ d8 / d16 displacement). **Base** = BX or BP · **Index** = SI or DI · **Displacement** = 8-bit (sign-extended) or 16-bit.

Defaults: BX/SI/DI/disp → **DS** · BP and SP → **SS** · IP → **CS** · String destination DI → **ES**.

Memory modes checklist: Direct (address in instruction) → Register indirect (address in register) → Based (BX/BP + disp) → Indexed (SI/DI + disp) → Based-index (both + disp) → String (SI/DI, DS/ES).

I/O: Direct = 8-bit port address in the instruction; Indirect = 16-bit port address in **DX** (only DX).

Brackets `[ ]` mean "contents of" — never the address itself.

Continue with Part 1 (Architecture) and Part 3 (Instruction Set) — they carry the remaining exam questions of Module 2.